Battleship AI Agent Project Proposal

CS 175: Project in AI, Spring 2026

Project Members

- **Ehsan Mahmud Shishir**, *eshishir*, 51169223
- **Vardan Keshishyan**, *vardank*, 91535149
- **Han Seo Ro**, *hsro*, 21607112

Main Idea

Our project idea is to build a Battleship game and design a computer strategy for playing it. In Battleship, players have a grid to place ships and take turns guessing where the opponent's ships are. The objective is to sink all the other player's ships before losing your own ships. In this project, we are particularly interested in the attacking strategy of the computer player. The computer should play Battleship, keep track of which moves have been made, use the results of previous moves, and decide what square to shoot next. We will see if a more intelligent computer player can play better than a basic player that mostly guesses randomly. For the user, the game should display the board, the moves, and which strategy performs better.

Input/Output

The program will be given the Battleship board configuration, size, ship positions and sizes, past moves, whether each attack was a hit or miss, and which ships have sunk. The AI agents will choose their move based on the current state. The baseline agent may only use the list of untried cells and previous hits, while the more advanced agents will also use the previous feedback and the amount of space to estimate where ships are likely to be. The program will output the next best move (row and column on the opponent's board). The entire system will also produce the outcome of each move, who won the game, total number of attacks, hits and misses, and other statistics after playing multiple games. This can be used as a project to compare AI agents in a simple game of hidden information. Thus, we are thinking of doing four agents based on random-guessing, checkerboard pattern guessing, probability heatmap, and Monte Carlo.

Basic approach, Method, and Algorithm

We plan on implementing four different types of agents as listed below:

Agent 1: Random guessing. This agent chooses every unattacked cell randomly and will be used as a baseline model to compare to.

Agent 2: Checkerboard. This agent alternates between global and local search. On a clean board, this agent picks a cell in a checkerboard pattern until it gets a hit, after which it proceeds to a local search and prioritizes nearby cells to determine the ship's orientation.

Agent 3: Probability heatmap. The probability heatmap agent maintains a probability distribution over all the cells on the board, where it keeps track of the probability of a ship being in a cell given the current observations. Misses will eliminate cells from consideration whereas hits will increase the probability of the neighboring cells. This agent updates its belief state after each observation and chooses a cell based on the highest probability.

Agent 4: Monte Carlo. Approximates the hidden board distribution by generating many possible legal enemy boards that are consistent with the current observations. It will then estimate the probability that a cell has a ship by counting the frequency of a ship occupying that cell across the sampled boards.

Evaluation plan

We will compare the performances of agents 2, 3, and 4 against the baseline agent 1 and look at quantitative metrics such as win rate, average number of moves needed to win, hit rate, runtime per move (efficiency), and overall performance across different board sizes and numbers of ships to evaluate their performance. Ideally, we expect to see our agents perform better across these metrics when compared to the random-guessing agent.

Project Milestones

Milestone 1: Terminal Based Battleship Environment

We will first build a terminal-based Battleship game without a frontend. The game will have different board sizes to choose and support legal ship placement. It will be a turn based game with various feedbacks such as miss, hit and ship destruction. The game should be playable in the terminal so that we have a reliable foundation before adding AI agents.

Milestone 2: Agents Implementation

After the terminal-based game environment is working, we will implement the main AI agents. We plan to start with a Random Agent that selects from unattacked cells without using strategy. Then we will implement a Checkerboard Agent, which searches using an alternating checkerboard pattern and switches to nearby targeting after a hit. Next, we will implement a Probability Heatmap Agent that updates a probability heatmap based on known hits, misses, sunk ships, and remaining ship sizes. Finally, we will implement a Monte Carlo Hybrid Agent that samples many possible enemy boards consistent with the current observations and uses those samples to estimate the best next move.

Milestone 3: Ship Placement Agent

We will implement agents that decide how ships are placed before the game begins. The first version will use random legal placement. Later versions may use more strategic placement, such as keeping the ships far from each other or placing ships in ways that are harder for the attacking agents to discover. This will allow us to test not only how well agents attack, but also how different placement strategies affect the difficulty of the game.

Milestone 4: Agent vs Agent Simulation

We will create a simulation mode where attacking agents play against ship-placement agents automatically. This will allow us to run many games without manual input and collect performance data. In the final version, two agents should be able to play against each other: one placing ships and the other trying to find and destroy them. This simulation mode will be the foundation for our evaluation.

Milestone 5: Agent Improvement

We will test the agents on different board sizes and different ship sizes and placement configurations. As the board size increases, we will adjust the number and size of ships accordingly. This will help us evaluate whether the agents remain effective as the search space becomes larger.

Milestone 6: Final Evaluation, Interface, and Report

After collecting the final results, we will finalize the agent simulation system, and prepare our final report. If there is enough time, we may also build a simple frontend that shows the board state, AI moves, hits and misses, ship destruction, and probability heatmaps.